

Cursul 6

Interfața în linia de comandă

Utilizarea Sistemelor de Operare (USO)

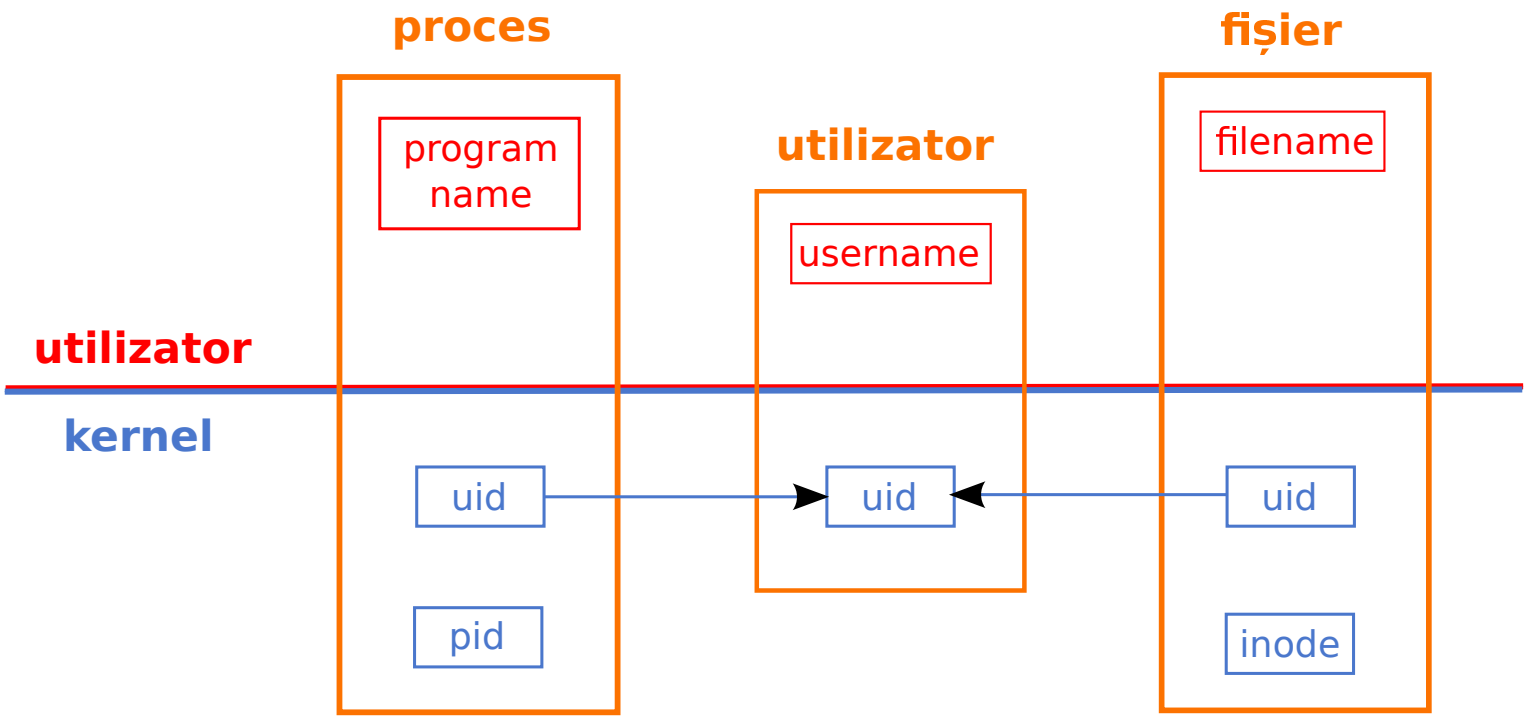
30 octombrie 2017

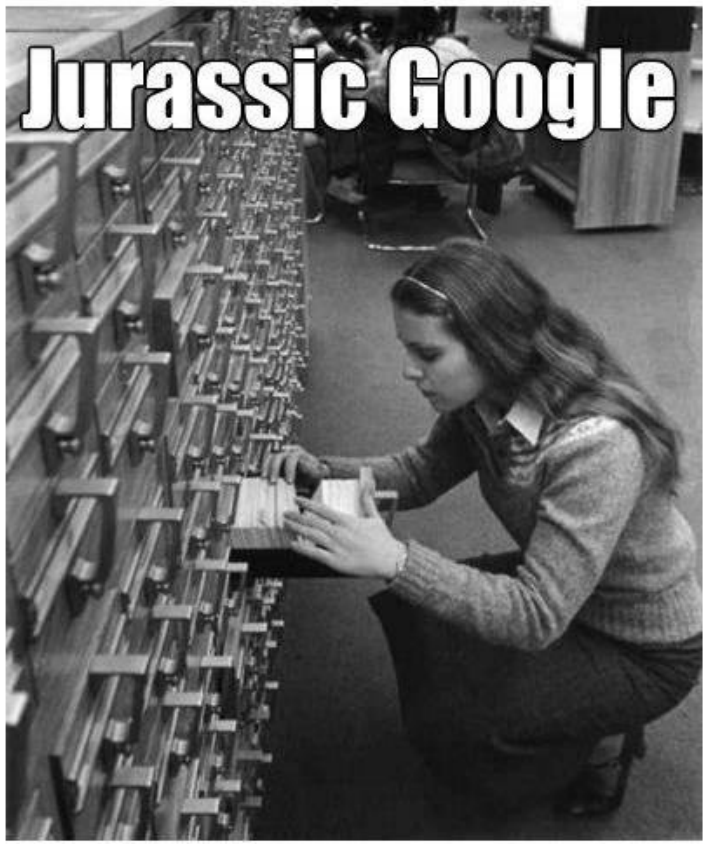
Departamentul de Calculatoare

Moto

It was a mistake to think that GUIs ever would, could, or even should, eliminate CLIs.

Jeffrey Snover (Architect of Windows PowerShell)


proces
program name
utilizator
username
fișier
filename
utilizator
kernel
uid
uid
uid
pid
inode



http://crazyhyena.com/funny-file-system-is-the-prehistoric-google_en

În această săptămână la USO

- weekly quest 4 în week end-ul 4-5 noiembrie 2017
- miercuri, 1 noiembrie 2017, începe laboratorul 6: Interfața în linia de comandă
- tema 2 apare pe parcursul acestei săptămâni
  - întrebați, întrebați, întrebați
  - folosiți forumul
  - ajutați colegii pe forum
  - `https://ocw.cs.pub.ro/courses/uso/regulament`
    - secțiunea ,,Recomandări de comunicare electronică”
    - secțiunea ,,Realizarea temelor”
- slide-urile cursului 7: ,,Servicii de rețea” vor fi publicate vineri, 3 noiembrie 2017

- Suport (Introducere în sisteme de operare)
  - Capitolul 12 – Shell scripting
    - Secțiunile 12.1, 12.2, 12.6, 12.7

Fie următoarele scenarii de utilizare/deziderate

- folosire eficientă a sistemului
- competențe de folosire a unui sistem, indiferent de sistemul de operare/distribuție
- administrare de sisteme de la distanță
- administrare a unei rețele complexe, cu echipamente de rețea dedicate
- proiectarea de piese în AutoCAD
- prelucrări numerice și statistice (Matlab, Octave, R)

Soluția este CLI. De ce?

- CLI: *Command Line Interface*
- oferă opțiuni de a rula rapid comenzi (combinații de taste, istoric de comenzi, completare automată)
- se pot înlănțui comenzi pentru a obține funcționalități noi
- se pot automatiza rapid acțiuni folosind scripting (*shell scripting* este compus din comenzi shell)
- majoritatea comenzilor sunt independente de distribuție, funcționează la fel pe Unix-uri (Linux, Mac OS X, FreeBSD, Android)
- flexbilitate maximă: o opțiune nouă în linia de comandă pentru o funcționlitate nouă
  - foarte multe aplicații oferă interfață în linia de comandă
  - unele aplicații oferă doar interfață în linia de comandă
- pentru administrare de la distanță: servere, echipamente dedicate

Unde folosim CLI?

- shell-ul unui sistem de operare, fie local, fie la distanță
- aplicații dedicate: Matlab, R, Python, AutoCAD
- console de jocuri
- echipamente de rețea dedicate

- interacțiunea utilizatorului cu aplicațiile și cu sistemul de operare
- ascunde detaliile sistemului de operare
- interacțiunea cu sistemul de calcul
- *Human Machine Interface*
- GUI – *Graphical User Interface*
  - WIMP (window, icon, menu, pointing device)
  - touchscreen
- CLI – *Command Line Interface*
  - execuția de comenzi pentru îndeplinirea sarcinilor

De ce GUI?

- *eye-candy*
- ușor de folosit, intuitivă
- ascunderea complexității
- personalizabilă (*customizable*)

De ce CLI?

- consecventă, arată peste tot la fel
- eficientă
- ușor de adăugat o opțiune nouă
  - un argument față de un buton nou
- automatizarea sarcinilor
- resurse consumate reduse

Cazuri de utilizare GUI și CLI

- GUI
  - multimedia
  - grafică 3D
  - jocuri
  - Internet browsing
- CLI
  - administrare sistem: utilizatori, sistem de fișiere, rețea, procese
  - scripting/automatizare, eficientizare
  - aplicații și dispozitive dedicate
- în general vom folosi ambele tipuri de interese, depinzând de nevoie

- consola GRUB (bootloader)
- consola Python
- consola de configurare de echipamente de rețea (Cisco IOS, Juniper JunOS)
- consola Matlab, AutoCAD, R
- consolă de jocuri (FPS)

Consola Python

```
user@host$ python
Python 2.7.8 (default, Oct  7 2014, 17:59:21)
[GCC 4.9.1] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> 4**9
262144
>>> hex(157)
'0x9d'
>>> 0x9d
157
```

Cum arată o interfață în linia de comandă?

- prompt
  - poate fi un caracter precum $, %, #, >
  - la prompt se introduc comenzi
- comenzile sunt șiruri de caractere
- numele comenzii oferă intuiție legat de ce face comanda
- comenzile pot avea argumente (opțiuni pentru comandă)
  - argumentele se separă prin spații

- depinzând de linia de comanda, este posibil un pas de autentificare
- se introduce comanda și argumentele
- se apasă ENTER
- interpretorul de comenzi analizează comanda și execută o acțiune
- interpretorul de comenzi face acțiunea dictată de comandă

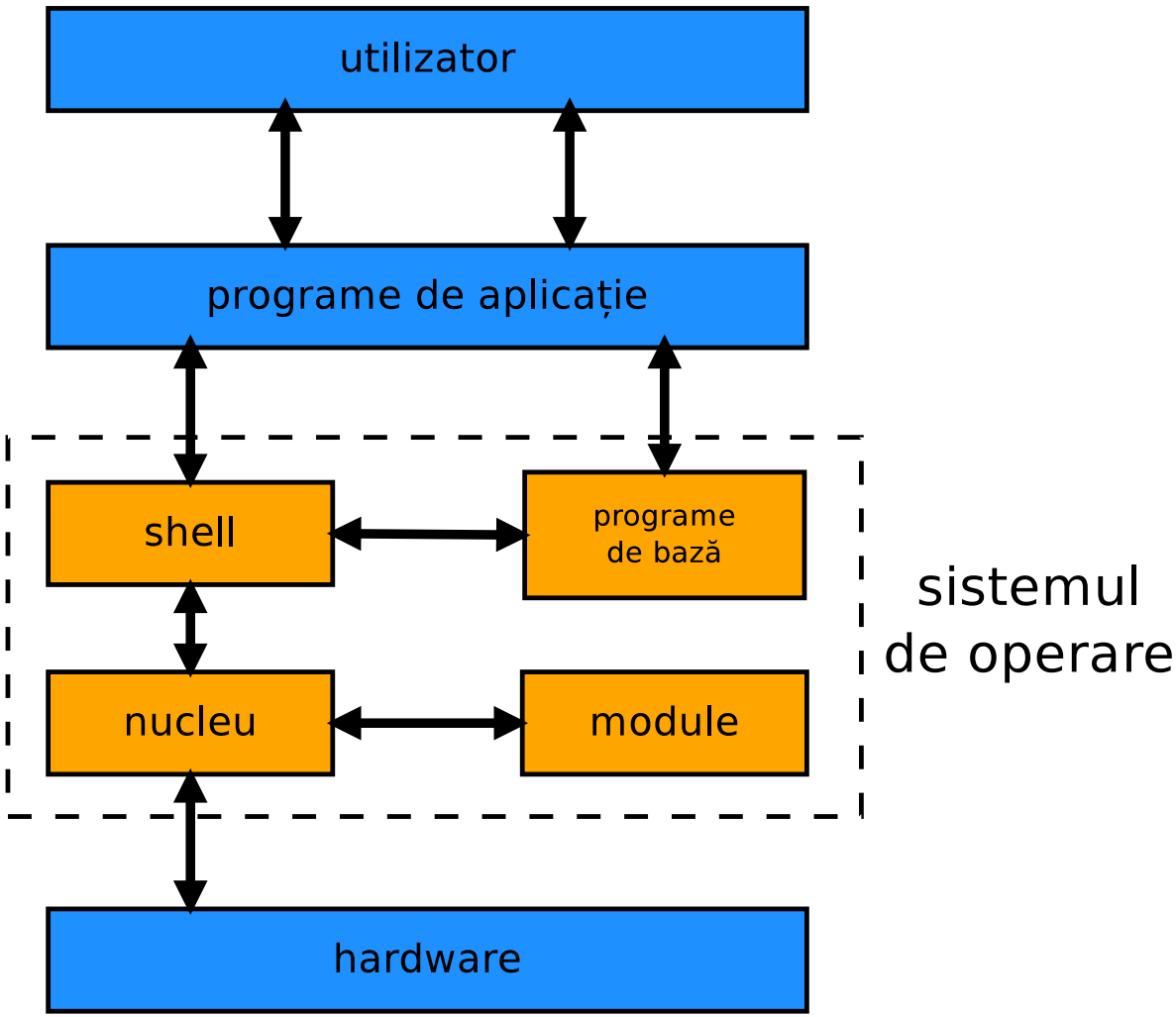
Facilități CLI

- command completion
- ajutor, documentare
- istoric de comenzi
- scurtături de taste pentru editare rapidă de comenzi: șterge cuvânt, mergi la început de comandă etc.

Shell

- aplicație care permite accesul la resursele sistemului de operare
- poate fi grafic (GUI) sau în linia de comandă (CLI)
- în Windows shell-ul este `Explorer` (GUI)
- în Unix
  - shell GUI: X Window System + KDE/GNOME/Xfce/Awesome/Fluxbox
  - shell-uri CLI: `bash`, `tcsh`, `zsh`, `dash`
- în general un shell este un caz particular de CLI
- Cum putem rula o sesiune de shell în Linux?
  - terminal virtual (`ALT+CTRL+F1...F6`)
  - emulator de terminal (`konsole`, `gnome-terminal`, `xterm` etc.)
  - conexiune de la distanță
- terminalul este (pseudo)dispozitivul în care rulează shell-ul
- în continuare, shell = shell CLI

- terminal = dispozitiv
- shell = proces
- terminal – cadru, interfață pentru rularea unui shell
  - fizic (tastatură, monitor)
  - virtual (ALT+CTRL+F1...F6) – tty0, tty1, ..., tty6
  - emulator de terminal – pts/0, pts/1, ...
  - folosit pentru intrarea/ieșirea standard (*stdin*/*stdout*) a unui proces
  - daemonii nu au acces la nici un terminal (nu au intrare/ieșire standard)
- shell – permite rularea de comenzi
  - prompt, comenzi, operatori shell
  - bash, zsh, tcsh


utilizator
programe de aplicație
shell
programe
de bază
nucleu
module
hardware
sistemul
de operare

Promptul unui shell

```
ana@host:~$ whoami
ana
```

- comenzile pot avea argumente (opțiuni pentru comandă)
  - argumentele se separă prin spații
- Keep it simple!
  - nume simple, uneori neintuitive: `cp`, `rm`, `ls`, `vi`, `ps`

- `command --help`: opțiuni și argumente ale comenzii
- `whatis command`: ce face comanda
- `apropos concept`: comenzi care au legătură cu acel concept
- `man command`: pagina de manual a comenzii
  - `q` pentru ieșire
  - `/` pentru căutare înainte, `?` pentru căutare înapoi
  - `n`: next match, `N`: previous match
- Google (,,tutorial", ,,faq", ,,how to", ,,example", ,,usage")

Funcționarea unui shell

- se introduce o comandă (`ls`)
- shell-ul identifică executabilul asociat (`/bin/ls`)
- este creat un nou process shell (`/bin/bash`) (`fork`)
- procesul creat se ,,transformă" conform cu noul executabil
- intrarea standard este controlată de noul proces
- după execuția comenzii, procesul se încheie și controlul revine shell-ului

- command completion
  - folosirea tastei `TAB` sau a apăsării de două ori `TAB TAB`
  - completarea comenzii și a argumentelor acesteia
- istoricul comenzilor
  - `history`: vizualizarea istoriei
  - `Ctrl+p` (sau tastă sus), `Ctrl+n` (sau tastă jos): anterioara și următoarea comandă
  - `Ctrl+r`: *reverse search*, căutare înapoi
- biblioteca readline (`libreadline`)
  - *keyboard shortcuts* (eficiență în utilizare)
  - documentare: `man bash` și căutare după `Commands for`

Redirectarea intrării/ieșirii: >, <

```
ana@host:~$ seq 1 100 > out.txt
ana@host:~$ wc -l < out.txt
100
```

Comunicarea între procese: |

```
ana@host:~$ svn log | less
```

Rularea unui proces în background: &

```
ana@host:~$ du -hs &
[1] 15766
ana@host:~$ jobs
[1]+  Running                 du -hs &
```

Extinderea comenzilor shell (cont.)

Secvențierea comenzilor: ;

```
root@asgard:~# apt-get update ; apt-get upgrade ; apt-get dist-upgrade
```

Secvențierea condițională: &&, ||

```
ana@host:~$ id ana &> /dev/null && echo "da" || echo "nu"
da
ana@host:~$ id atilla &> /dev/null && echo "da" || echo "nu"
nu
```

Comenzi utile în shell-ul Unix

- head, tail: afișează doar primele/ultimele linii din fișier
- wc: numără liniile din fișier
- grep: extrage liniile care conțin o anumită expresie
- tr: transformă caractere în alte caractere
- cut: extrage doar anumite coloane
- sort: sortează liniile primite la intrare
- uniq: unicizează liniile de la intrare (elimină duplicatele); intrările trebuie să fie sortate
- seq: generează numere între anumite intervale

Redirectare și comunicarea între procese

- în cazul redirectării, operatorii sunt $>$ (redirectarea ieșirii standard), $<$ (redictarea intrării standard) și $2>$ (redirectarea ieșirii de eroare standard)
- la redirectare forma este `command operator file`
- spunem că redirectăm într-un/dintr-un fișier
- în cazul comunicării între procese forma este `command1 | command2`
- spunem că redirectăm ieșirea standard a comenzii `command1` în intrarea standard a comenzii `command2`

Ce efect are comanda de mai jos?

```
ana@host:~$ < /dev/urandom tr -d -c 'A-Za-z0-9' | head -c 8 ; echo
```

- `/dev/urandom` – generator de caractere aleatoare
- `< /dev/urandom` – redictează intrarea standard din fișierul `/dev/urandom`
- `tr` – *transliterate* (comandă shell de tip **filtru de text**)
- `-d` – delete (șterge)
- `-c` – complement (set complementar)
- `-c 'A-Za-z0-9'` – orice nu este literă sau cifră

→ șterge orice **nu** este literă sau cifră dintr-un șir infinit generat aleator

- `head` – extrage primele elemente
- `-c 8` – opt caractere

→ extrage primele 8 caractere

→ extrage primele 8 caractere tipăribile (literă sau cifră) dintr-un șir generat aleator; la final afișează un caracter *newline*

One liner

- o înlănțuire de mai multe comenzi shell
- încape pe o singură linie în shell
- oferă o nouă funcționalitate
- folosește operatori de redirectare și de secvențiere
- quick'n'dirty: scriem comenzi înlănțuite și le rulăm
- pentru funcționalități avansate e recomandată scrierea de scripturi shell

Caractere speciale shell

- operatori
- redirectare: >, <, &>, >>, <<
- secvențiere, înlănțuire: ;, ||, &&, |, &
- expandare: $
- comentare: #
- citare (escaping): ', ", \
- separare: blank (spațiu)
- globbing: ?, *, [,], {, }

- reținerea anumitor valori/informații (în variabile)
- afișarea caracterelor speciale
- operații pe mai multe intrări dintr-o dată

Inițializare variabile în shell

```
ana@host:~$ a=10
ana@host:~$ b=ana
ana@host:~$ c="ana are mere"
ana@host:~$ d="$b $a"
ana@host:~$ echo $a, $b, $c, $d
10, ana, ana are mere, ana 10
```

- **nu** se folosește spațiu înainte și după egal la inițializare
  - s-ar considera comandă cu argumente
- folosirea valorii unei variabile se realizează prin prefixarea cu simbolul $ (*dollar*)
- variabilele shell **nu** au tip (întreg, șir, pointer)

Variabile speciale shell

- documentație: `man bash`, și căutare după *Special Parameters*
- `$?` – valoarea de retur a ultimei comenzi
- `$$` – PID-ul procesului shell curent
- `$!` – PID-ul ultimului proces (job) lansat în background
- `$_` – ultimul argument al ultimei comenzi

- definesc contextul de rulare a unui proces
- sunt moștenite de procesele copil
- exemple: `PATH`, `HOME`, `PWD`, `USERNAME`, `SHELL`
- operații specifice cu variabile de mediu
  - listare: `env`, `printenv`
  - configurare variabilă ca variabilă de mediu (exportare)
    - `export JAVA_HOME`
    - `export JAVA_HOME=/home/student/sun-java-jdk7 # init and export`
  - persistența configurării (*startup configuration*)
    - în fișierul `~/.bashrc`

Citare (escaping)

Exemple de escaping

```
ana@host:/tmp/uso$ ls -l
total 0
-rw-r--r-- 1 ana ana 0 2007-11-18 00:14 alfa beta.txt
-rw-r--r-- 1 ana ana 0 2007-11-18 00:14 d.txt
ana@host:/tmp/uso$ ls alfa beta.txt
ls: alfa: No such file or directory
ls: beta.txt: No such file or directory
ana@host:/tmp/uso$ ls "alfa beta.txt"
alfa beta.txt
ana@host:/tmp/uso$ ls 'alfa beta.txt'
alfa beta.txt
ana@host:/tmp/uso$ ls alfa\ beta.txt
alfa beta.txt
ana@host:/tmp/uso$ echo $a
4
ana@host:/tmp/uso$ echo "$a"
4
ana@host:/tmp/uso$ echo \$a
$a
ana@host:/tmp/uso$ echo '$a'
$a
```

Citare (escaping) (cont.)

- escaping folosind \ (*backslash*)
  - forțează forma literală a caracterului următor
  - rol special în cazul comenzii echo -e

```
ana@host:~$ echo -e "a\tb\tc"
a       b       c
```

- escaping folosind " (ghilimele, *quote*)
  - escapează textul dintre ghilimele
  - nu escapează expandarea

```
ana@host:~$ echo "* ; && | > $sir"
* ; && | > file.txt
```

- ’ (apostrof, *single quote*)
  - escapează orice între apostroafe

```
ana@host:~$ echo '\\\$a$bc$de'
\\\$a$bc$de
```

Expandarea variabilelor

```
ana@host:-$ echo $PATH
/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games:/usr/local/PacketTracer5
ana@host:-$ echo $PATH | tr ':' '\n'
/usr/local/bin
/usr/bin
/bin
/usr/local/games
/usr/games
/usr/local/PacketTracer5
```

Expandarea comenzilor

```
ana@host:~$ ls
ana@host:~/tmp$ touch $(seq -f "file-%02g.txt" 1 20)
ana@host:~/tmp$ touch $(seq -f "file-%02g.txt" 1 20)
ana@host:~$ ls
file-01.txt  file-03.txt  file-05.txt  file-07.txt  file-09.txt  file-11.txt  file-13.txt  [...]
file-02.txt  file-04.txt  file-06.txt  file-08.txt  file-10.txt  file-12.txt  file-14.txt  [...]
```

Expandarea aritmetică

```
ana@host:~$ a=3
ana@host:~$ ((a++))
ana@host:~$ b=$((a+1))
ana@host:~$ echo $b
5
```

Folosire globbing în comenzi shell

```
ana@host:~/code/tests$ ls
Makefile  endian.c  struct_bit    struct_bit.o  tcp_new_sock.c
endian    endian.o  struct_bit.c  tcp_new_sock  tcp_new_sock.o
ana@host:~/code/tests$ ls *.c
endian.c  struct_bit.c  tcp_new_sock.c
ana@host:~/code/tests$ ls end?an.c
endian.c
ana@host:~/code/tests$ ls [a-s]*.c
endian.c  struct_bit.c
ana@host:~/code/tests$ ls *.{c,o}
endian.c  endian.o  struct_bit.c  struct_bit.o  tcp_new_sock.c  tcp_new_sock.o
```

Globbing (cont.)

- pentru a rula o comandă pe mai multe fișiere
- filename expansion (și altele)
- `*` – orice caracter de oricâte ori
- `?` – orice caracter
- `[abcde]`, `[a-e]` – set de caractere
- `[^abc]` – diferit de setul de caractere
- `{jpg,png,bmp}` – selecție între variante

Facilități CLI și shell

- prompt, comenzi, argumente
- command completion
- ajutor, documentare
- istoric de comenzi
- scurtături de taste
- redirectare
- comunicare între procese
- one liner
- variabile, variabile de mediu
- escaping
- globbing

- interfață
- GUI
- CLI
- prompt
- comandă, argumente
- command completion
- istoric de comenzi
- shell
- terminal

- documentare
- libreadline
- >, <, &, |, ||, &&, ;
- ”, ’, \,$
- one liner
- variabile
- variabile de mediu
- escaping
- expandare
- globbing

Resurse utile

- http://en.wikipedia.org/wiki/User_interface
- http://www.gnu.org/software/bash/manual/bashref.html
- http://mywiki.wooledge.org/BashFAQ
- http://www.shelldorado.com/
- http://www.pixelbeat.org/cmdline.html

- ediția a 3-a, 2002
- enciclopedie a faciltăților Unix
- 1100 de pagini, 51 de capitole
- pentru “power users” – utilizatori avansați care vor să folosească la maxim sistemul
- capitole excelente despre shell și shell scripting

Rob Pike



- parte a echipei Unix
- ulterior, dezvoltator la Plan 9 și Inferno
  - sisteme de operare unde totul este un fișier
  - pe Unix aproape totul este un fișier
- cărți și proiecte împreună cu Ken Thompson, Brian Kernighan
- UTF-8
- lucrează la Google

Amazon

- Amazon.com, Inc.
- comerț electronic (online retailer)
- fondată în 1994, online în 1995
- inițial librărie online
- Amazon Kindle
- în 2010, vânzările de e-books au depășit vânzările de cărți în format printat
- Amazon Web Services – servicii de stocare și rulare, cloud computing
  - Amazon EC2 – închiriere de mașini virtuale
  - Amazon S3 – servicii de stocare

- CLI pentru Windows
- rulează de la Windows XP SP2 în sus
- integrare cu .NET framework
- *cmdlets*, aplicații .NET
- automatizare – scripting
- pipeline de obiecte (*objects*), nu de șiruri de caractere ca în Unix